By: Tyler Bish

Table of Contents

Page #

1. Project Functionality Overview 2
2. Repository Directory 3
   i) Repository Location 3
   ii) Repository Structure 4
3. Environment Setup 5
   i) Cloning Repository for Local Development 5
   ii) Necessary Software and Packages 5
   iii) Necessary Environment Variables 6
   iv) Database Directory Setup 7
4. Modifying Existing Features 8
   i) Web Application 8
      a) Home Page 8
      b) Admin Page 9
      c) Intake Form Pages 11
   ii) Authentication 12
      a) User Authentication 12
      b) Admin Authentication 14
   iii) Email Sending 15
   iv) PDF Generation 16
   v) Monday.com Integration 18
   vi) Security and Data Logging 22
5. Adding New Features 23
6. Production vs Development Environments 24
   i) Production Branch Considerations 24
   ii) Development Branch Considerations 25
7. Docker Containerization Support 26
   i) Dockerfile 26
   ii) Docker-compose.yml (local development) 26
   iii) Docker Deployment 28
8. Google Cloud Platform (GCP) Support 28
   i) CI/CD Workflow 28
   ii) Secrets Manager 30
   iii) Maintenance and Monitoring 31

By: Tyler Bish

Project Functionality Overview

This project is a Django web application used to manage and automate the collection of business and individual client intake information. The application locks intake form access behind temporary credentials, stores permitted submission data, generates PDF copies of completed forms, emails users through Microsoft Graph, and syncs submitted forms and relevant contact information into the Monday.com CRM.

The application has four major functional areas:

1. **Client intake access**

   A client must authenticate with a temporary login ID and password before accessing either the business intake form or the individual intake form. These temporary credentials are stored in the database, have an expiration time, and are marked as used after the form workflow is completed.

2. **Intake form capture**

   The application collects business and personal intake information through Django forms. Forms validate input such as SSNs, ZIP codes, phone numbers, dates, ownership percentages, and certification fields.

3. **Document generation**

   The application fills PDF templates with validated form data. The PDF templates are stored in the repository and are filled by the pypdf package. Output PDFs are written to generated_forms/.

4. **External communication and synchronization**

   The system sends credential and confirmation emails to clients through Microsoft Graph API. It also creates Monday.com board items and uploads generated PDFs to Monday.com file columns via the Monday.com embedded GraphQL API.

Important privacy behavior includes:

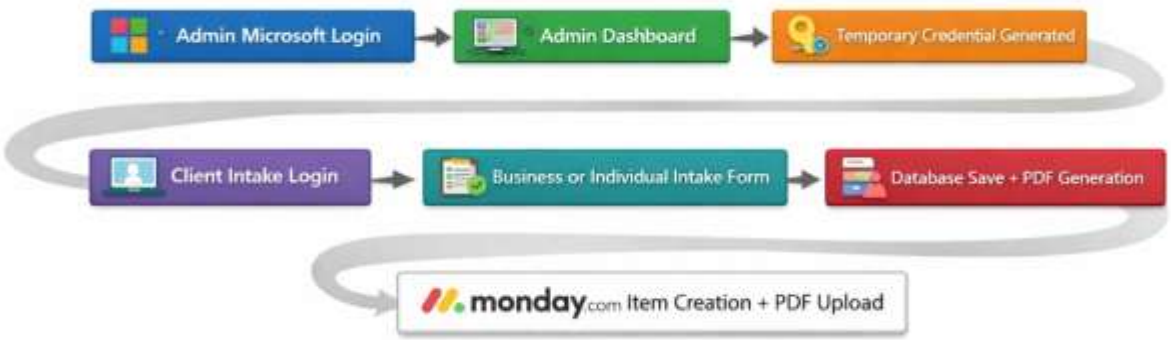
- SSNs are accepted and validated by forms, but they are not stored in the database.
- Bank account numbers are accepted and validated for business intake PDF generation, but they are not stored in the database.
- Generated PDFs may contain sensitive client information and should be treated as runtime artifacts.

The primary workflow for this project is as follows:

1. The web application is hosted locally via Docker deployment or publicly via Google Cloud.
2. Admin user signs in with Microsoft authentication.
3. Admin creates a temporary credential for a business or individual intake.
4. Client receives the credential by email.
5. Client logs in and completes the assigned intake form.
6. Application validates and saves non-sensitive submission data.
7. Application generates a completed PDF from the PDF template.
8. Application creates a Monday.com board item.
9. Application uploads the generated PDF to the Monday.com file column.

By: Tyler Bish

Workflow Visual Diagram:



The system intentionally separates form validation from database persistence. Some sensitive values are collected and used only during the request. For example, SSNs and bank account numbers may be needed for PDF generation, but the current models do not store them in the database.

Repository Directory

The repository is organized as a Django project with one main local application: src.apps.core. Most business logic is in the core app. Project-level configuration, including settings and URL routing, is located under src/config.

Repository Location

The repository remote currently points to:

https://github.com/tjbish/InsightersIncWorkflowAutomation.git

Clone command:

```
git clone https://github.com/tjbish/InsightersIncWorkflowAutomation.git
cd InsightersIncWorkflowAutomation
```

To verify the remote after cloning:

```
git remote -v
```

Expected output should include:

```
origin  https://github.com/tjbish/InsightersIncWorkflowAutomation.git (fetch)
origin  https://github.com/tjbish/InsightersIncWorkflowAutomation.git (push)
```

Development should be performed from the repository root. The repository root is the directory that contains manage.py, requirements.txt, Dockerfile, and docker-compose.yml

By: Tyler Bish

Repository Structure

How to read:

- ***FILENAME*** → **DESCRIPTION**

Root-level files and folders:

- *manage.py* → Django command entry point.
- *requirements.txt* → Python dependency list.
- *Dockerfile* → Container build definition for the application.
- *docker-compose.yml* → Local development container orchestration.
- *compose.debug.yaml* → Docker Compose override for debugpy-based debugging.
- *cloudbuild.yaml* → Google Cloud Build pipeline for building, migrating, and deploying to Cloud Run.
- *scripts/* → Deployment and container helper scripts.
- *documentation/* → Documentation artifacts for easy handoff.
- *generated_forms/* → Runtime output directory for generated PDFs. Files in this directory may contain sensitive client information and should not be treated as source code.

Project configuration:

- *src/config/settings/base.py* → Shared settings for local and production environments.
- *src/config/settings/local.py* → Local development overrides.
- *src/config/settings/production.py* → Production settings, Google Secret Manager integration, Static file injector WhiteNoise, and secure cookie/HTTPS settings.
- *src/config/urls.py* → Project-level URL routing. Includes the core app, Django admin, and allauth routes.

Core application:

- *src/apps/core/models.py* → Database models for business submissions, personal submissions, and temporary credentials.
- *src/apps/core/forms.py* → Django form definitions, validators, labels, choices, and widgets.
- *src/apps/core/views.py* → Main request workflow logic: login, dashboard, form submission, PDF generation calls, email calls, and Monday.com synchronization.
- *src/apps/core/urls.py* → Core application route definitions.
- *src/apps/core/email.py* → Microsoft Graph email helpers.
- *src/apps/core/pdf_mapping.py* → Mapping from Django form fields to PDF template field names.
- *src/apps/core/pdf_engine.py* → Low-level PDF template loading and AcroForm field writing.
- *src/apps/core/pdf_templates/* → Source PDF templates: BusinessForm.pdf and IndividualForm.pdf.

Templates and tests:

- *src/templates/* → HTML templates for pages, intake forms, admin dashboard, processing page, and email bodies.
- *src/static/* → Images and static assets.
- *src/apps/core/tests/* → Django unit tests for forms and PDF generation.
- *tests/* → Request-level pytest tests that interact with a running local server.

By: Tyler Bish

Environment Setup

Local setup can be performed directly on the host machine or through Docker. The host-machine setup is useful for code editing, quick Django commands, and unit tests. Docker setup is useful for matching the containerized runtime and running PostgreSQL without installing it directly on the host

Cloning Repository for Local Development

Use Git to clone the repository:

```
git clone https://github.com/tjbish/InsightersIncWorkflowAutomation.git
cd InsightersIncWorkflowAutomation
```

After cloning, check the repository status:

```
git status
```

Navigate to the desired branch before making changes:

```
git checkout branch_name
```

Recommended local workflow:

1. Pull or clone the latest repository.
2. Create a virtual environment.
3. Install dependencies.
4. Create the .env file.
5. Run migrations.
6. Start the development server.
7. Run tests before and after code changes.

Necessary Software and Packages

Required host software:

- Git
- Python 3.10 or newer
- Pip
- PowerShell or another terminal

Recommended host software:

- Docker Desktop
- PostgreSQL client tools
- Visual Studio Code or another IDE

Python virtual environment setup:

```
py -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
pip install -r requirements.txt
```

By: Tyler Bish

Important Python packages from requirements.txt:

- *Django>=4.2,<5.0* → Main web framework.
- *gunicorn>=21.2.0* → Production WSGI server.
- *psycopg2-binary>=2.9.9* → PostgreSQL database adapter.
- *django-environ>=0.11.2* → Environment variable parsing and .env loading.
- *requests>=2.31.0* → HTTP requests for Microsoft Graph and Monday.com APIs.
- *google-cloud-secret-manager>=2.16.0* → Google Secret Manager integration for production.
- *django-bootstrap5* → Bootstrap template integration.
- *pypdf>=4.2.0* → PDF template filling.
- *django-allauth==0.61.1* → Microsoft authentication integration.
- *whitenoise>=6.5.0* → Static file serving in production.

Run a basic Django configuration check:

```
python manage.py check
```

Necessary Environment Variables

The application loads a .env file from the repository root. This behavior is defined in src/config/settings/base.py:

```
BASE_DIR = Path(__file__).resolve().parent.parent.parent.parent
env = environ.Env()
environ.Env.read_env(os.path.join(BASE_DIR, '.env'))
```

Minimum local .env example:

```
DJANGO_SETTINGS_MODULE=src.config.settings.local
DJANGO_SECRET_KEY=<unique-django-secret>
DEBUG=True
ALLOWED_HOSTS=localhost,127.0.0.1
BASE_URL=http://localhost:8000
INTAKE_LOGIN_ID=admin
INTAKE_LOGIN_PASSWORD=admin
ENTRA_CLIENT_ID=<microsoft-client-id>
ENTRA_CLIENT_SECRET=<microsoft-client-secret>
ENTRA_TENANT_ID=<microsoft-tenant-id>
MONDAY_API=<prod-Monday-api-key>
DEV_MONDAY_API=<dev-Monday-api-key>
MONDAY_BOARD_ID=<prod-Monday-board-id>
DEV_MONDAY_BOARD_ID=<dev-Monday-board-id>
GOOGLE_CLOUD_PROJECT=insighters-app
```

- INTAKE_LOGIN_ID / PASSWORD are local-only bypass credentials to test intake forms.
- Monday.com API keys and board IDs are different for production and development deployment.

Optional local PostgreSQL database url:

```
DATABASE_URL=postgres://postgres:postgres@localhost:5432/insighters_db
```

- If DATABASE_URL is omitted, current settings fall back to: sqlite:///tmp/db.sqlite3

Microsoft authentication and email variables:

- ENTRA_CLIENT_ID
- ENTRA_CLIENT_SECRET
- ENTRA_TENANT_ID

Monday.com variables:

- MONDAY_API → Production Monday.com API key.
- DEV_MONDAY_API → Development Monday.com API key.
- MONDAY_BOARD_ID → Production Monday.com Board ID.
- DEV_MONDAY_BOARD_ID → Development Monday.com Board ID.

Google Cloud production variable:

- GOOGLE_CLOUD_PROJECT → Links name of google cloud project to active code repository.

Cloud Run deployment variables supplied by cloudbuild.yaml:

```
GOOGLE_CLOUD_PROJECT=$PROJECT_ID
DJANGO_SETTINGS_MODULE=src.config.settings.production
ALLOWED_HOSTS=*
DEV_MONDAY_BOARD_ID=$_MONDAY_BOARD_ID
ENTRA_CLIENT_ID=$_ENTRA_CLIENT_ID
ENTRA_TENANT_ID=$_ENTRA_TENANT_ID
DATABASE_CONNECTION_NAME=$_DB_CONNECTION_NAME
BASE_URL=https://insighters-workflow-automation-428824878696.us-
central1.run.app
```

Database Directory Setup

The project supports two local database patterns:

1. SQLite fallback

   If DATABASE_URL is not configured, base.py defaults to sqlite:///tmp/db.sqlite3. This is sufficient for lightweight local development and Django unit tests.

2. PostgreSQL

   Docker Compose starts a PostgreSQL service named db and configures the web container with:

   ```
   DATABASE_URL=postgres://postgres:postgres@db:5432/insighters_db
   ```

   For host-machine PostgreSQL, use localhost instead of the Docker service name:

   ```
   DATABASE_URL=postgres://postgres:postgres@localhost:5432/insighters_db
   ```

Apply migrations:

```
python manage.py migrate
```

Create migrations after changing models:

```
python manage.py makemigrations
python manage.py migrate
```

Docker database lifecycle:

```
docker compose up –build
docker compose down
```

To delete the local Docker PostgreSQL volume:

```
docker compose down -v
```

- Use docker compose down -v only when local database contents can be discarded.

Modifying Existing Features

Existing features should be modified by following the path of the data through the application. A small user-facing change may require updates to a form, a template, a view, tests, PDF mapping, email context, and Monday.com mappings.

The representative code blocks in this section are included to show the working pattern used by the application, not to duplicate every line of source code. When a code example appears here, read it as a pointer to the responsible layer: the route shows how a request enters the feature, the form or validator shows where user input is constrained, the view shows where workflow decisions happen, and the settings or helper code shows which external service the feature depends on. This is important because many features cross several files. Changing only the visible template may update the page but leave PDF output, Monday.com sync, or data persistence unchanged.

Web Application

The web application is composed of Django views and templates. Routes are defined in src/apps/core/urls.py, view functions are defined in src/apps/core/views.py, and templates are stored in src/templates/.

General modification guide:

1. Change page layout in templates.
2. Change request behavior in views.py.
3. Change validation in forms.py.
4. Change persisted data in models.py.
5. Change generated PDFs in pdf_mapping.py or pdf_engine.py.

Home Page

The home page route is:

```
path("", views.home, name="home")
```

The home page HTML template is located at:

```
src/templates/home.html
```

The view is intentionally simple:

```
def home(request):
      return render(request, "home.html")
```

How and why this works:

The empty string route makes the home page the root URL of the application. Django calls the home view when a browser requests /. The view does not query the database or build custom context because the home page is currently static. Keeping this view small is intentional: it reduces the chance that a simple public page can fail because an unrelated database, email, or API dependency is unavailable.

To modify the home page:

1. Edit src/templates/home.html.
2. If new context data is required, update the home view in views.py.
3. If new static assets are required, place them under src/static/.
4. Run the development server and verify http://localhost:8000/.

Example context expansion:

```
def home(request):
      return render(
          request,
          "home.html",
          {"support_email": "support@example.com"},
      )
```

Only add view context when the template needs dynamic values. Static content should remain in the template.

Admin Page

The admin dashboard route is:

```
path("dashboard/", views.admin_dashboard, name="admin_dashboard")
```

The dashboard HTML template is located at:

```
src/templates/admin.html
```

The dashboard is protected by Django authentication in views.py:

```
@login_required(login_url="admin_login")
def admin_dashboard(request):
...
```

By: Tyler Bish

How and why this works:

The login_required decorator prevents unauthenticated visitors from reaching the credential generation workflow. The explicit login_url points users to the project admin login page instead of relying on Django's default login route. This matters because the dashboard can create temporary client credentials and trigger emails, so it should only be reachable after admin authentication has succeeded.

The dashboard creates temporary credentials for user login with a randomly generated password:

```
generated_password = secrets.token_urlsafe(12)
credential = TemporaryIntakeCredential.objects.create(
      login_id=login_id,
      password_hash=make_password(generated_password),
      form_type=form.cleaned_data["form_type"],
      client_email=form.cleaned_data["client_email"],
      created_by_login_id=created_by_login_id,
      expires_at=expires_at,
  )
```

How and why this works:

The password is generated with secrets.token_urlsafe so the value is random and suitable for a login secret. The raw generated password is shown or emailed once, but the database receives password_hash instead of the plain text password. The form_type value ties the credential to either the business or individual workflow, and expires_at keeps old credentials from remaining valid indefinitely. created_by_login_id provides an audit trail back to the admin who generated the credential.

To modify admin credential behavior:

1. Update TemporaryIntakeCredentialCreateForm in forms.py if inputs change.
2. Update admin_dashboard in views.py if creation behavior changes.
3. Update admin.html if the displayed fields or messages change.
4. Preserve password hashing with make_password.
5. Avoid logging generated_password.

The dashboard also checks for a Microsoft token:

```
if not SocialToken.objects.filter(
      account__user=request.user,
      account__provider='microsoft'
  ).exists():
      messages.warning(...)
```

This warning matters because email sending depends on Microsoft Graph tokens. Admin authentication proves who the admin is, but email sending also needs a stored SocialToken from the Microsoft provider. If that token does not exist, the dashboard can still load, but credential email delivery through Microsoft Graph may fail.

By: Tyler Bish

Intake Form Pages

The intake form routes are:

```
path("business/", views.business_view, name="business_intake")
path("individual/", views.personal_view, name="individual_intake")
```

The intake form HTML templates are located at:

```
src/templates/business_intake.html
src/templates/personal_intake.html
```

Both views are protected by temporary intake authentication in views.py:

```
@require_intake_login
def business_view(request):
    ...

@require_intake_login
def personal_view(request):
    ...
```

Business form files:

- src/apps/core/forms.py → BusinessIntakeForm
- src/templates/business_intake.html → business intake template
- src/apps/core/models.py → BusinessIntakeSubmission
- src/apps/core/pdf_mapping.py → business PDF mapping

Individual form files:

- src/apps/core/forms.py → PersonalIntakeForm
- src/templates/personal_intake.html → individual intake template
- src/apps/core/models.py → PersonalIntakeSubmission
- src/apps/core/pdf_mapping.py → individual PDF mapping

Representative form field validator:

```
phone_validator = RegexValidator(
      regex=r'^\d{3}-\d{3}-\d{4}$',
      message='Enter a valid phone number (e.g., 555-555-5555).'
  )
```

How and why this works:

RegexValidator runs as part of Django form validation before the view saves a submission, generates a PDF, or sends data to Monday.com. The ^ and $ anchors require the entire input to match the pattern, not just part of it. The \d{3} groups require exactly three digits, the hyphens require the displayed 555-555-5555 format, and \d{4} requires the final four digits. This predictable format is useful because the same cleaned phone value is later converted for Monday.com phone columns and PDF output. If the desired format changes, update the validator and then check every downstream consumer of that field.

Representative sensitive field:

```
client_ssn = forms.CharField(
      label="SSN*",
      max_length=11,
      min_length=11,
      validators=[ssn_validator],
      widget=forms.PasswordInput(
          attrs={'placeholder': 'XXX-XX-XXXX', 'autocomplete':'off'},
          render_value=True
      )
  )
```

How and why this works:

The SSN field is collected through a Django form because the PDF template needs the value, but the corresponding model intentionally does not store SSNs. The min_length and max_length constraints require the exact XXX-XX-XXXX length, and ssn_validator enforces the specific pattern. PasswordInput hides the value on screen so it is not casually exposed while typing, autocomplete is disabled to reduce browser retention of sensitive data, and render_value=True preserves the field during a validation error so the user does not have to re-enter it after another field fails. This is a usability tradeoff and should be handled carefully: never log this value, never add it to model fields, and avoid adding it to session data unless the workflow explicitly requires it.

When modifying intake pages, decide whether the field is:

- display-only,
- validated but not stored,
- stored in the database,
- written to the generated PDF,
- synchronized to Monday.com,
- included in email content.

Do not add SSNs or bank account numbers to database models unless the security

requirements are formally changed.

Authentication

The project has two authentication systems:

1. Client intake authentication → Uses TemporaryIntakeCredential records.
2. Admin authentication → Uses Django authentication and Microsoft allauth.

These systems are intentionally separate.

User Authentication

Client users authenticate with temporary intake credentials. The credential fields generated by the web-app and stored in the database include:

- login_id

- password_hash
- form_type
- client_email
- created_by_login_id
- expires_at
- used_at

Representative credential validation:

```
def is_valid_for_login(self):
    now = timezone.now()
    return self.used_at is None and self.expires_at > now
```

How and why this works:

Temporary credentials are one-time, time-limited access records rather than permanent user accounts. The used_at check prevents reuse after the workflow is completed, and the expires_at comparison prevents stale credentials from remaining valid. timezone.now() is used instead of a naive datetime so the comparison works with Django's timezone-aware settings.

Representative login logic:

```
credential =
TemporaryIntakeCredential.objects.filter(login_id=login_id).first()

if credential is None:
     return None

expected_form_type = _path_to_form_type(next_path)if next_path else None

if expected_form_type and credential.form_type != expected_form_type:
    return None

if not credential.is_valid_for_login():
    return None

if not check_password(password, credential.password_hash):
    return None
```

How and why this works:

The lookup starts with login_id because that is the public identifier sent to the client. The form_type check prevents a credential generated for one intake type from opening the other form. is_valid_for_login centralizes the expiration and one-time-use rules on the model. check_password compares the submitted password against the stored hash without exposing or recovering the original password. Returning None for each failure keeps the caller from revealing whether the login ID, password, form type, or expiration status was the reason access failed.

By: Tyler Bish

User authentication modification guidance:

- Keep credentials form-scoped.
- Keep credentials expiration-based.
- Keep password storage hashed.
- Keep local bypass credentials limited to development.
- Avoid creating permanent client user accounts unless the workflow changes.

Admin Authentication

Admin authentication is handled by Django and django-allauth with Microsoft as a provider. Relevant settings are in src/config/settings/base.py:

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    ...
    'allauth',
    'allauth.account',
    'allauth.socialaccount',
    'allauth.socialaccount.providers.microsoft',
]

AUTHENTICATION_BACKENDS = [
    'django.contrib.auth.backends.ModelBackend',
    'allauth.account.auth_backends.AuthenticationBackend',
]

LOGIN_REDIRECT_URL = '/dashboard'
LOGIN_URL = '/admin/login/'
```

How and why this works:

Django's built-in authentication backend supports normal admin/session behavior, while the allauth backend adds Microsoft sign-in. LOGIN_REDIRECT_URL sends authenticated admins to the dashboard, and LOGIN_URL tells protected views where to send unauthenticated users. These settings are global, so changes here affect more than one page.

By: Tyler Bish

Microsoft provider configuration:

```
SOCIALACCOUNT_PROVIDERS = {
      'microsoft': {
          'TENANT': MICROSOFT_TENANT_ID,
          'APP': {
              'client_id': MICROSOFT_CLIENT_ID,
              'secret': MICROSOFT_CLIENT_SECRET,
          },
          'SCOPE': [
              'User.Read',
              'Mail.Send',
              'offline_access',
          ],
      }
  }
```

How and why this works:

The Microsoft provider settings connect django-allauth to the Entra app registration. User.Read allows the sign-in identity to be read, Mail.Send is required because the application sends credential and confirmation emails through Microsoft Graph, and offline_access allows allauth to store refreshable tokens. Removing scopes may not break login immediately, but it can break email sending later when the application tries to call Graph.

Admin authentication modification guidance:

- Preserve Mail.Send scope if email sending remains part of the system.
- Preserve offline_access if stored tokens are still needed.
- Update Microsoft Entra app registration when redirect URLs change.
- Verify /accounts/ routes after allauth changes.

Email Sending

Email sending is implemented in:

```
src/apps/core/email.py
```

Email HTML templates are located at:

```
src/templates/email_temp.html src/templates/email_confirmation.html
```

Credential email function:

```
send_intake_email(request, credential_id, plain_password)
```

Submission confirmation function:

```
send_submission_confirmation_email(submission, credential,
pdf_path=None)
```

Representative Microsoft Graph request:

```
endpoint = "https://graph.microsoft.com/v1.0/me/sendMail"

response = requests.post(
    endpoint,
    headers={
        "Authorization": f"Bearer {token.token}",
        "Content-Type": "application/json"
    },
    json=payload
)
```

How and why this works:

Email is sent through Microsoft Graph instead of Django's SMTP backend because the workflow depends on the authenticated Microsoft account and its Graph permissions. The endpoint targets the signed-in user's mailbox. The bearer token comes from the SocialToken stored by allauth, and the JSON payload contains the message body, recipients, and subject built from the email template. A successful Graph send returns HTTP 202, which means the message was accepted for delivery rather than synchronously delivered to the recipient's inbox.

Email modification guidance:

- Modify email_temp.html for intake credential emails.
- Modify email_confirmation.html for completed submission emails.
- Modify email.py if templates need additional context variables.
- Do not log tokens, passwords, or full payloads containing sensitive data.
- Confirm that Graph returns HTTP 202 for successful sends.

Common email failure causes:

- Admin did not sign in through Microsoft.
- SocialToken record is missing.
- Microsoft token lacks Mail.Send.
- Recipient address is invalid.
- Microsoft Entra app configuration changed.

PDF Generation

PDF generation is split into mapping and engine layers.

Mapping layer code is located at:

```
src/apps/core/pdf_mapping.py
```

Engine layer code is located at:

```
src/apps/core/pdf_engine.py
```

By: Tyler Bish

Template PDF files are located at:

```
src/apps/core/pdf_templates/BusinessForm.pdf
src/apps/core/pdf_templates/IndividualForm.pdf
```

Representative individual mapping:

```
DIRECT_TEXT_FIELDS = {
    "tax_year": "tax_year",
    "client_name": "client_name",
    "spouse_name": "spouse_name",
    "client_dob": "client_dob",
    "client_ssn": "client_ssn",
    "email": "email",
    "client_signature": "client_signature",
    "date_signed": "date_signed",
}
```

How and why this works:

The mapping dictionary connects cleaned Django form field names to the internal field names embedded inside the PDF template. In many cases both names are the same, but the mapping still matters because it gives the application one place to adapt when a PDF template field has a different name, a typo, or a naming convention that should not leak back into forms.py. Add PDF-only field changes here instead of changing model names just to match a template.

Representative text conversion:

```
def _to_text(value: Any) -> str:
    if value is None:
        return ""
    if isinstance(value, (datetime, date)):
        return value.strftime("%m/%d/%Y")
    if isinstance(value, Decimal):
        return format(value, "f")
    return str(value)
```

How and why this works:

PDF form fields expect text values. The conversion helper normalizes Python values before they are written into the PDF: empty values become blank strings, dates use the expected month/day/year display format, Decimal values are converted without scientific notation, and everything else falls back to str(). Keeping this conversion in the PDF layer prevents display formatting from changing how values are stored in the database.

By: Tyler Bish

Representative PDF engine call:

```
def fill_individual_pdf(cleaned_data, *, output_path,
template_path=INDIVIDUAL_TEMPLATE):
    field_values = build_individual_pdf_field_values(cleaned_data)
    return _fill_pdf(field_values, output_path=output_path,
template_path=template_path)
```

How and why this works:

The fill function receives cleaned form data, builds the PDF-specific field dictionary, and then delegates the low-level writing to the shared engine. This separation keeps business rules and field naming in pdf_mapping.py while keeping pypdf mechanics in pdf_engine.py. When a PDF value is wrong, check the mapping first. When the PDF cannot be written or opened correctly, check the engine and template.

Representative field writing:

```
for page in writer.pages:
      writer.update_page_form_field_values(
          page,
          field_values,
          auto_regenerate=True,
      )
```

How and why this works:

The engine iterates through every page because PDF form fields can exist on any page of the template. update_page_form_field_values writes the prepared values into matching AcroForm fields. auto_regenerate=True asks PDF viewers to refresh field appearances so the filled values are visible when the generated document is opened.

PDF modification guidance:

- Use pdf_mapping.py when the displayed PDF value needs to change.
- Use pdf_engine.py when the PDF writing mechanics need to change.
- Do not change model values to solve PDF-only formatting issues.
- Confirm PDF template internal field names before changing mappings.
- Some template field names include typos and are intentionally mapped as-is.
    - Notably: bank_account_type in business pdf is misspelled bank_accuont_type

Monday.com Integration

Monday.com integration automatically accesses an existing board in the Monday.com CRM using the Monday GraphQL API and uploads the email, phone number, form creation date, and generated PDF form. This functionality is currently implemented in src/apps/core/views.py.

Key functions in views.py:

- _to_monday_column_values
    - Converts a local submission payload into properly formatted Monday.com column values, handling special types like email and phone.
- _build_business_monday_column_values

    - Builds the column values dictionary specifically for business submissions using predefined monday column mappings.
- _monday_create_item
    - Sends a GraphQL request to Monday.com to create a new board item with the given name and column values.
- _monday_upload_file_to_column
    - Uploads a generated PDF file to a Monday.com file column for a specific item.
- _sync_monday_submission_with_file
    - Orchestrates the full Monday sync process by creating the item (if needed) and uploading the associated PDF file.

Runtime settings used by the Monday.com helpers:

- MONDAY_API_TOKEN API → token used in the Authorization header.
- MONDAY_BOARD_ID → Board where created items are written.
- MONDAY_BUSINESS_GROUP_ID → Group used for business intake items.
    - Current default: group_mm262dz.
- MONDAY_PERSONAL_GROUP_ID → Group used for personal intake items.
    - Current default: topics.
- MONDAY_FILE_ID File column used when uploading the generated PDF.
    - Current default: file_mm1nffry.
- MONDAY_BUSINESS_COLUMN_MAP → Maps local business payload fields to Monday.com column IDs.
    - Current default includes:
        - email -> email_mm26e82x
        - phone_number -> phone_mm26ghe3.
- MONDAY_PERSONAL_COLUMN_MAP → Maps local personal payload fields to Monday.com column IDs.
    - Current default includes:
        - email -> email_mm26e82x
        - phone_number -> phone_mm26ghe3
        - date_signed -> date4.

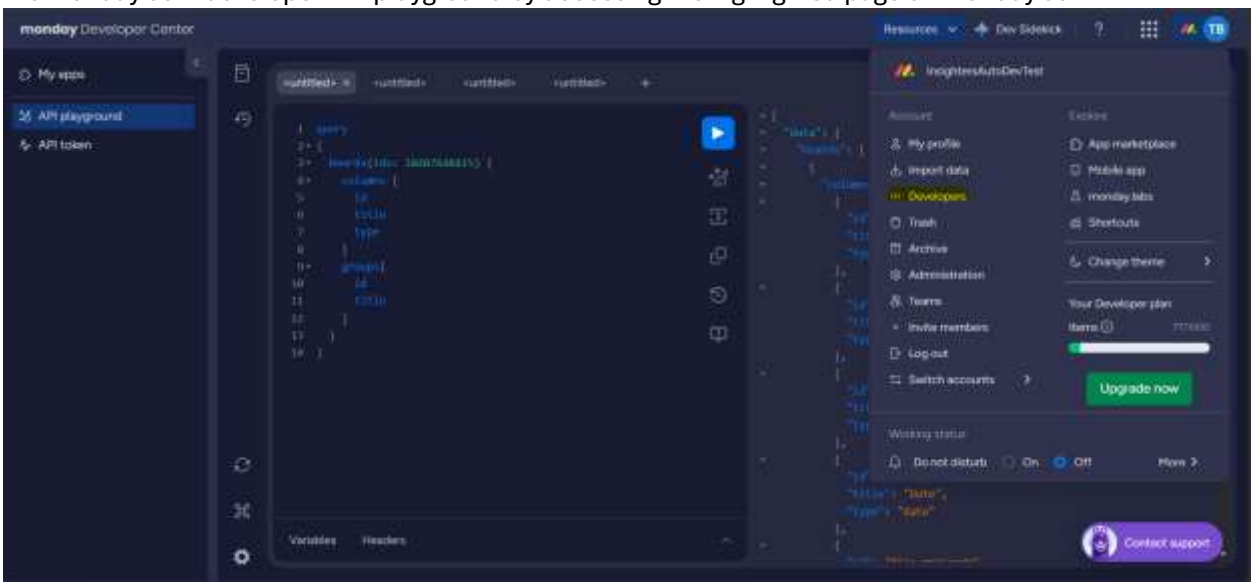
Development board settings kept in base.py:

- DEV_MONDAY_API_TOKEN
- DEV_MONDAY_BOARD_ID
- DEV_MONDAY_BUSINESS_GROUP_ID
- DEV_MONDAY_PERSONAL_GROUP_ID
- DEV_MONDAY_FILE_ID
- DEV_MONDAY_BUSINESS_COLUMN_MAP
- DEV_MONDAY_PERSONAL_COLUMN_MAP

These DEV_MONDAY_* values are useful for testing against development boards, but the current Monday.com helper functions do not read them directly. To test with development board values without changing code, set the runtime MONDAY_* environment variables to the development token, board, group, file column, and column IDs.

By: Tyler Bish

To find the required field mappings for a new Monday.com board or for additional new fields, navigate to the Monday.com developer API playground by accessing the highlighted page on Monday.com:



Once there, you can use the API playground to quickly query your Monday.com environment to get all board IDs using the query below:

```
query {
  boards {
    id
    name
    groups {
      id
    }
  }
}
```

Once you know the necessary board ID, use the query below to get the required field mappings:

```
query
{
  boards(ids: YOUR_BOARD_ID) {
    columns {
      id
      title
      type
    }
    groups{
      id
      title
    }
  }
}
```

By: Tyler Bish

Representative email column mapping:

```
column_values[monday_column_id] = {"email": value, "text": value}
```

How and why this works:

Monday.com email columns do not behave like plain text columns. They expect a structured JSON object with both an email value and display text. Passing only a string can appear to work for simple text columns but fail or produce invalid data for typed Monday.com columns.

Representative phone column mapping:

```
column_values[monday_column_id] = { "phone":
_extract_phone_number(value), "countryShortName": "US" }
```

How and why this works:

Monday.com phone columns also expect structured JSON. The helper extracts a clean phone number from the validated form value and pairs it with a country code. The current implementation assumes U.S. clients, so international phone support would require both validation changes in forms.py and payload changes in the Monday.com helper.

Representative item creation mutation:

```
mutation = """
  mutation ($board_id: ID!, $group_id: String!, $item_name: String!,
            $column_values: JSON!) {
    create_item(
    board_id: $board_id,
    group_id: $group_id,
    item_name: $item_name,
    column_values: $column_values
    ) {
    id
    name
    }
  }
"""
```

How and why this works:

The create_item mutation creates the Monday.com row after the form is submitted and the submission payload has been prepared. board_id selects the destination board, group_id selects the section of that board, item_name becomes the row name, and column_values carries the mapped form data. The helper sends variables separately from the GraphQL mutation text so IDs and payload values are not manually interpolated into the query string.

Monday.com modification guidance:

- Confirm board ID, group ID, and column IDs before changing mappings.
- Confirm which board is the active target before deployment.
- Email and phone columns require structured JSON payloads.

- Generic text columns can usually accept simple strings.
- File upload requires the generated PDF path to exist.
- Development and production boards may use different column IDs.
- Keep Monday.com tokens out of logs, commits, Cloud Build substitutions, and template output.

Security and Data Logging

Security-sensitive files:

- src/apps/core/forms.py
- src/apps/core/views.py
- src/apps/core/email.py
- src/apps/core/error_filters.py
- src/config/settings/production.py

Sensitive data rules:

- Do not commit .env.
- Do not log SSNs.
- Do not log bank account numbers.
- Do not log generated passwords.
- Do not log Microsoft tokens.
- Do not log Monday.com tokens.
- Treat generated PDFs as sensitive output.

Representative sensitive decorators:

```
@sensitive_post_parameters('fin_number', 'bank_account_number',
'bank_routing_number')
@sensitive_variables()
@require_intake_login def business_view(request): ...
```

How and why this works:

sensitive_post_parameters tells Django's error reporting system to mask named POST fields if an exception page or error report is generated. sensitive_variables masks local variables in tracebacks from the decorated function. These decorators are safeguards for failures, not permission to handle sensitive data casually. They should be kept close to views that receive SSNs, FINs, bank account numbers, routing numbers, generated passwords, or API-related payloads.

Production logging includes a dynamic filter in production.py that redacts known secrets, SSNs, emails, and bearer tokens. This is defensive logging behavior and should not replace careful application code. Developers should still avoid printing sensitive values.

By: Tyler Bish

Adding New Features

Adding a new feature should begin with identifying the affected workflow. Most features fall into one of these categories:

- New page
- New form field
- New validation rule
- New stored data
- New PDF output
- New email content
- New Monday.com sync field
- New admin dashboard capability

General feature checklist:

1. Add or update route in src/apps/core/urls.py.
2. Add or update view in src/apps/core/views.py.
3. Add or update form in src/apps/core/forms.py.
4. Add or update template in src/templates/.
5. Add or update model in src/apps/core/models.py if storage is required.
6. Create migrations for model changes.
7. Add or update PDF mapping if the value belongs in a generated PDF.
8. Add or update email context/templates if notifications change.
9. Add or update Monday.com mappings if board sync changes.
10. Add or update tests.

Example: adding a new personal intake field.

Model:

```
preferred_contact_method = models.CharField(
    max_length=20,
    blank=True,
    null=True
)
```

Form:

```
CONTACT_METHODS = [
    ("email", "Email"),
    ("phone", "Phone"),
    ("text", "Text Message"),
]

preferred_contact_method = forms.ChoiceField(
    label="Preferred Contact Method",
    choices=CONTACT_METHODS,
    required=False
)
```

View persistence:

```
preferred_contact_method=data.get("preferred_contact_method") or None
```

PDF mapping:

```
DIRECT_TEXT_FIELDS = {..."preferred_contact_method":
"preferred_contact_method"}
```

Migrations:

```
python manage.py makemigrations
python manage.py migrate
```

Verification:

```
python manage.py check
python manage.py test
```

The key development principle is to keep each behavior in the layer where it belongs. A PDF-only display change belongs in PDF mapping. A validation change belongs in forms.py. A stored-data change belongs in models.py and the view that creates model instances.

Production vs Development Environments

The application has different expectations in development and production. Development favors fast local iteration using Docker containerization to deploy to https://localhost/8000. Production favors secure secret handling, HTTPS behavior, cloud database access, static file serving, and structured logging and utilizes Google Cloud Run to deploy to https://insighters-workflow-automation-428824878696.us-central1.run.app/.

Production Branch Considerations

Production uses:

- DJANGO_SETTINGS_MODULE=src.config.settings.production

Production settings:

- DEBUG is False.
- Secret values are fetched from Google Secret Manager.
- DATABASES may be reconfigured after fetching DATABASE_URL.
- Microsoft provider secrets are updated after secret retrieval.
- Cloud Run receives non-secret deployment settings from cloudbuild.yaml.
- Secure cookies are enabled. - HTTPS redirect behavior may be enabled.
- WhiteNoise is inserted into middleware.
- Logging redacts sensitive values.

Representative production secret mapping:

```
SECRETS_MAPPING = {
      "SECRET_KEY": "DJANGO_SECRET_KEY",
      "DATABASE_URL_VAL": "DATABASE_URL",
      "MICROSOFT_CLIENT_SECRET": "ENTRA_CLIENT_SECRET",
      "DEV_MONDAY_API_TOKEN": "MONDAY_DEV_API", }
```

Production change guidance:

- Do not rely on local .env values for production-only secrets.
- Confirm Google Secret Manager values before deployment.
- Confirm Cloud Run environment variables.
- Confirm the deployed BASE_URL matches the active Cloud Run service URL because credential emails build absolute links from settings.BASE_URL.
- Confirm ALLOWED_HOSTS and CSRF_TRUSTED_ORIGINS.
- Confirm database migrations are safe to run against production data.
- Confirm Microsoft and Monday.com credentials point to production resources when production behavior is required.
- The Microsoft client secret comment in base.py notes a production expiration date of October 1, 2026. Refresh the Entra secret before that date.

Development Branch Considerations

Development uses:

- DJANGO_SETTINGS_MODULE=src.config.settings.local

Local settings:

- DEBUG = True
- ALLOWED_HOSTS = ['*']
- EMAIL_BACKEND = 'django.core.mail.backends.console.EmailBackend'

Development behavior:

- Local .env is loaded from the repository root.
- SQLite can be used if DATABASE_URL is omitted.
- Docker Compose can provide PostgreSQL.
- Intake login bypass can be configured with INTAKE_LOGIN_ID and INTAKE_LOGIN_PASSWORD.
- Monday.com development board values can be kept in DEV_MONDAY_* variables, but current production sync code reads MONDAY_* runtime settings.

Development change guidance:

- Keep local secrets in .env.
- Do not commit generated PDFs.
- Do not commit local databases.
- Use development Monday.com boards for integration testing, and map their values into MONDAY_* runtime settings when exercising the current sync code.
- Run tests before merging into production-facing branches.

By: Tyler Bish

Docker Containerization Support

Docker support allows the application to run in a consistent Linux container and provides a local PostgreSQL database through Docker Compose. For more detailed information see the Deployment and Architecture Documentation.

Dockerfile

The Dockerfile uses python:3.11-slim and installs project dependencies from requirements.txt.

Representative Dockerfile structure:

```
FROM python:3.11-slim
ENV PYTHONDONTWRITEBYTECODE=1
ENV PYTHONUNBUFFERED=1

RUN addgroup --system appgroup && adduser --system --ingroup appgroup
appuser

WORKDIR /app

RUN apt-get update && apt-get install -y \
    libpq-dev \
    gcc \
    && rm -rf /var/lib/apt/lists/*

COPY requirements.txt /app/
RUN pip install --no-cache-dir -r requirements.txt

COPY --chown=appuser:appgroup . /app/

ENTRYPOINT ["/app/scripts/entrypoint.sh"]

CMD ["sh", "-c", "exec gunicorn --bind :$PORT --workers 1 --threads 8
     --timeout 0 src.config.wsgi:application"]
```

Important Dockerfile behavior:

- Runs as a non-root app user.
- Installs system packages needed by PostgreSQL dependencies.
- Uses scripts/entrypoint.sh before starting Gunicorn.
- Exposes port 8080 for Cloud Run-style deployment.

docker-compose.yml (local development)

docker-compose.yml defines local web and database services. Detailed information is available in the Deployment and Architecture Documentation.

By: Tyler Bish

Representative web service:

```
web:
    build: .
    command: sh -c "python manage.py migrate &&
                    python manage.py runserver 0.0.0.0:8000"
    volumes:
      - .:/app
      - /app/staticfiles
    ports:
      - "8000:8000"
    depends_on:
      db:
        condition: service_healthy
    env_file:
      - .env
    environment:
      -
DATABASE_URL=postgres://postgres:postgres@db:5432/insighters_db
      - DJANGO_SETTINGS_MODULE=src.config.settings.local
```

Representative database service:

```
db:
    image: postgres:15
    ports:
      - "5432:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data
    environment:
      - POSTGRES_DB=insighters_db
      - POSTGRES_USER=postgres
      - POSTGRES_PASSWORD=postgres
```

Common Docker Compose commands:

```
docker compose up -build
docker compose down
docker compose down -v
docker compose exec web
python manage.py shell
docker compose exec web
python manage.py test
```

By: Tyler Bish

Docker Deployment

The production container starts through scripts/entrypoint.sh:

```
#!/bin/bash
set -e

echo "Collecting static files..."
python manage.py collectstatic --noinput

exec "$@"
```

Deployment behavior:

- Static files are collected before the server starts.
- Gunicorn starts the Django WSGI application.
- Environment variables determine whether production or local settings are used.
- Cloud Run supplies the PORT environment variable.

Docker deployment checklist:

- Build image successfully.
- Confirm requirements install successfully.
- Confirm collectstatic succeeds.
- Confirm migrations run before production traffic is served.
- Confirm production environment variables exist.
- Confirm secrets are available to the deployed runtime.

Google Cloud Platform (GCP) Support

GCP support is implemented through Cloud Build, Cloud Run, Google Secret Manager, and Cloud SQL integration. The deployment configuration is primarily in cloudbuild.yaml and src/config/settings/production.py. Detailed GCP deployment and configuration information is available in the Deployment and Architecture Documentation.

CI/CD Workflow

cloudbuild.yaml defines the deployment pipeline.

The pipeline performs these steps:

1. Build the Docker image.
2. Push the image to Artifact Registry.
3. Run Django database migrations through Cloud SQL connectivity.
4. Deploy the image to Cloud Run.

By: Tyler Bish

Current substitution defaults in cloudbuild.yaml:

```
_REGION: us-central1
_REPO_NAME: insighters-repo
_SERVICE_NAME: insighters-workflow-automation
_DB_CONNECTION_NAME: insighters-app:us-central1:insighters-prod-db
```

Representative build step:

```
- name: 'gcr.io/cloud-builders/docker'
    args:
      - 'build'
      - '-t'
      - '$_REGION-
docker.pkg.dev/$PROJECT_ID/$_REPO_NAME/$_SERVICE_NAME:$COMMIT_SHA'
      - '.'
```

Representative migration step:

```
- name: 'gcr.io/google-appengine/exec-wrapper'
    args:
      - '-i'
      - '$_REGION-
docker.pkg.dev/$PROJECT_ID/$_REPO_NAME/$_SERVICE_NAME:$COMMIT_SHA'
      - '-s'
      - '$_DB_CONNECTION_NAME'
      - '-e'
      - 'GOOGLE_CLOUD_PROJECT=$PROJECT_ID'
      - '-e'
      - 'DJANGO_SETTINGS_MODULE=src.config.settings.production'
      - '--'
      - 'python'
      - 'manage.py'
      - 'migrate'
      - '--noinput'
```

Representative Cloud Run deployment environment:

```
--set-env-vars
  GOOGLE_CLOUD_PROJECT=$PROJECT_ID,
  DJANGO_SETTINGS_MODULE=src.config.settings.production,
  ALLOWED_HOSTS=*,
  DEV_MONDAY_BOARD_ID=$_MONDAY_BOARD_ID,
  ENTRA_CLIENT_ID=$_ENTRA_CLIENT_ID,
  ENTRA_TENANT_ID=$_ENTRA_TENANT_ID,
  DATABASE_CONNECTION_NAME=$_DB_CONNECTION_NAME,
  BASE_URL=https://insighters-workflow-automation-428824878696.us-
central1.run.app
```

By: Tyler Bish

CI/CD modification guidance:

- Keep migration execution before Cloud Run deployment.
- Confirm substitution values match the target project.
- Avoid hardcoding secrets in cloudbuild.yaml.
- Confirm Cloud SQL instance names before changing database settings.
- Keep tokens and passwords in Secret Manager, not Cloud Build substitutions.
- Keep BASE_URL synchronized with the deployed service URL used by clients.

Secrets Manager

Detailed Secrets Manager information can be found in the Deployment and Architecture Documentation. Production secrets are fetched from Google Secret Manager in:

- src/config/settings/production.py

Representative helper:

```
def get_secret(secret_id: str, version_id: str = "latest") -> str:
    project_id = os.environ.get("GOOGLE_CLOUD_PROJECT")
    if not project_id:
        raise ValueError("GOOGLE_CLOUD_PROJECT environment variable
must be set.")
    name =
f"projects/{project_id}/secrets/{secret_id}/versions/{version_id}"
    response = client.access_secret_version(name=name)
    secret_value = response.payload.data.decode("UTF-8")
    return secret_value
```

Representative mapping:

```
SECRETS_MAPPING = {
    "SECRET_KEY": "DJANGO_SECRET_KEY",
    "DATABASE_URL_VAL": "DATABASE_URL",
    "MICROSOFT_CLIENT_SECRET": "ENTRA_CLIENT_SECRET",
    "DEV_MONDAY_API_TOKEN": "MONDAY_DEV_API",
}
```

Secrets Manager guidance:

- Secret IDs must match the names in SECRETS_MAPPING.
- GOOGLE_CLOUD_PROJECT must be set.
- Cloud Run service account must have permission to access secrets.
- Do not print secret values during debugging.
- Rotate Microsoft and Monday.com secrets according to organizational policy.

By: Tyler Bish

Maintenance and Monitoring

Production maintenance should focus on database migrations, secret health, authentication integrations, Monday.com integrations, generated-file handling, and logs.

Maintenance checklist:

- Confirm Cloud Run revisions are healthy after deployment.
- Confirm migrations completed successfully.
- Confirm Google Secret Manager values are current.
- Confirm Microsoft Entra client secret has not expired.
- Confirm Monday.com tokens and board IDs are current.
- Confirm generated PDFs are handled according to retention expectations.
- Review logs for errors without exposing sensitive data.

Monitoring considerations:

- Failed credential emails may indicate Microsoft token or permission issues.
- Failed Monday.com sync may indicate token, board, group, column, or file upload issues.
- Failed PDF generation may indicate template field or filesystem issues.
- Failed login attempts may indicate expired credentials or incorrect form type routing.

Representative production logging filter behavior in src/config/settings/production.py:

```
log_message = re.sub(
    r'\b\d{3}-\d{2}-\d{4}\b',
    '[REDACTED SSN]',
    log_message
)

log_message = re.sub(
    r'Bearer\s+[a-zA-Z0-9\-\._~\+\/]+',
    'Bearer [REDACTED_TOKEN]',
    log_message
)
```

The logging filter is a safeguard. Developers should still avoid logging sensitive values in the first place.